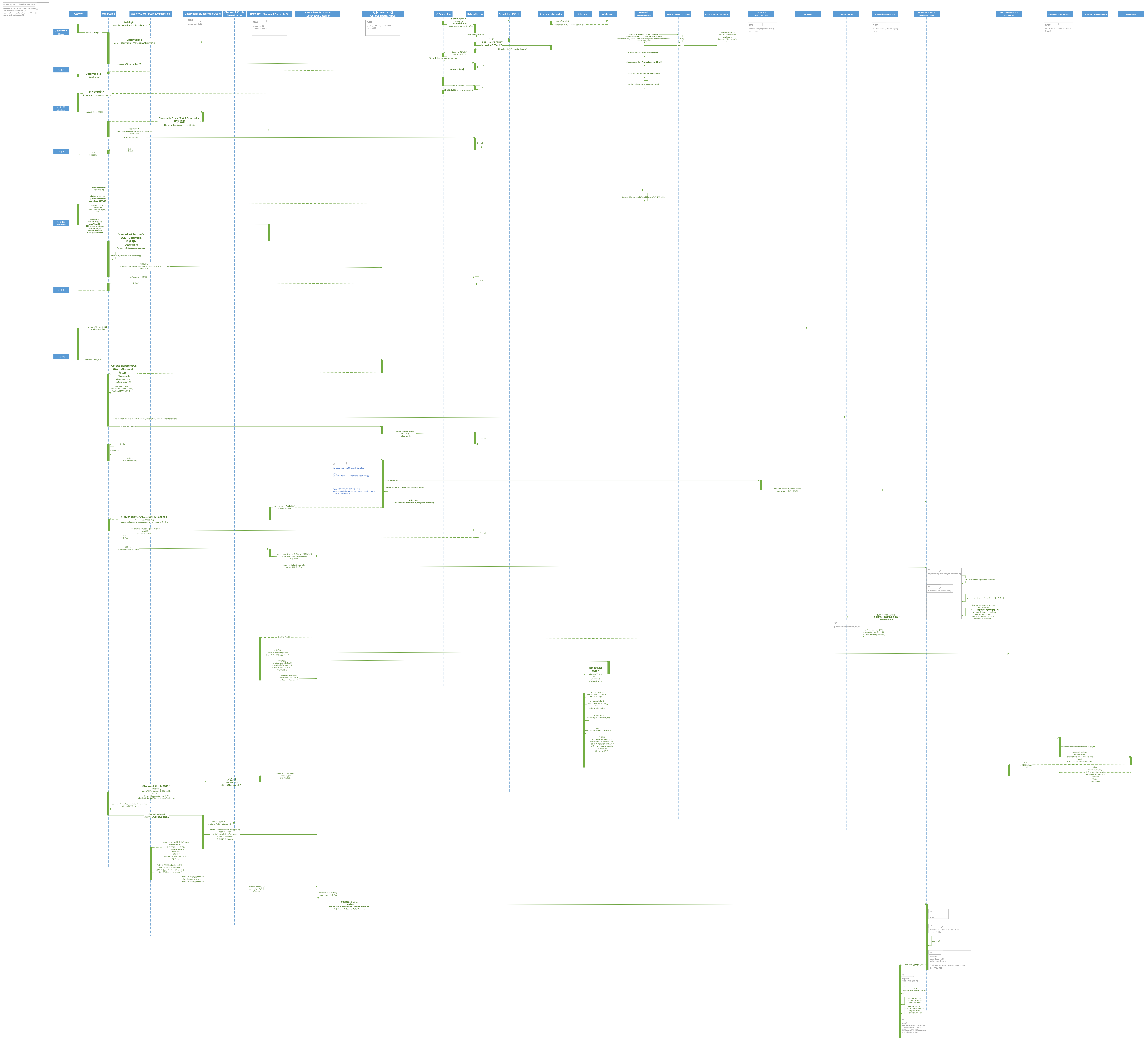
sc-shifo-RxJava3.0.12源码分析-2021-01-06
Observe.create(new ObservableOnSubscribe())
.subscribeOn(Schedulers.IO())
.observeOn(AndroidSchedulers.mainThread())
.subscribe(new Consumer())
Activity
Observable
Activity$1:ObservableOnSubscribe
Observable$1:ObservableCreate
ObservableCreate.CreateEmitter
对象1的$1:ObservableSubscribeOn
ObservableSubscribeOn.SubscribeOnObserver
对象2$1:RxJava包 ObservableObserveOn
IO:Schedulers
RxJavaPlugins
Schedulers.IOTask
Schedulers.IoHolder
Scheduler
IoScheduler
RxAndroid包 AndroidSchedulers
AndroidSchedulers$1:Callable
AndroidSchedulers.MainHolder
RxAndroid包 HandlerScheduler
Consumer
LambdaObserver
RxAnroid包HandlerWorker
ObservableObserveOn.ObserveOnObserver
ObservableSubscribeOn.SubscribeTask
IoScheduler.EventLoopWorker
IoScheduler.CachedWorkerPool
ThreadWorker
构造器
source = Activity$1
构造器
source = 对象1
scheduler = io调度器
构造器
scheduler = MainHolder.DEFAULT;
source = 对象2
标题
handler = Looper.getMainLooper()
async = true
构造器
handler = Looper.getMainLooper()
async = true
构造器
theadWorker = CachedWorkerPool池.get()
Activity#$1 = new ObservableOnSubscribe<T>
Observable 的create
create(Activity#$1)
Observable$1 = new ObservableCreate<>(Activity#$1)
onAssembly(Observable$1)
对象1
Observable$1
Schedulers.io()
Schedulers$7 = new $7Task()
Scheduler IO = RxJavaPlugins.initIoScheduler($7);
callRequireNonNull($7)
$7.get()
IoHolder.DEFAULT
IoHolder.DEFAULT
new IoScheduler()
Scheduler DEFAULT = new IoScheduler()
Scheduler DEFAULT = new IoScheduler()
Scheduler IO = new IoScheduler()
f == null
Observable$1
onIoScheduler(IO)
f == null
Scheduler IO = new IoScheduler()
返回io调度器
Scheduler IO = new IoScheduler()
对象1的 subscribeOn
subscribeOn(io调度器)
ObservableCreate继承了Observable, 所以调用 Observable#subscribeOn(io调度器)
对象1的$1 = new ObservableSubscribeOn<>(this, scheduler) this = 对象1
onAssembly(对象1的$1)
f == null
返回 对象1的$1
对象2
返回 对象1的$1
AndroidSchedulers.mainThread()
RxAndroidPlugins.onMainThreadScheduler(MAIN_THREAD)
返回MAIN_THREAD, 即AndroidSchedulers.MainHolder.DEFAULT = new HandlerScheduler(new Handler(Looper.getMainLooper()), true)
AndroidSchedulers$1 = new Callable();
AndroidSchedulers$1.call = MainHolder.DEFAULT
Scheduler MAIN_THREAD = RxAndroidPlugins.initMainThreadScheduler(AndroidSchedulers$1)
call()
DEFAULT
Scheduler DEFAULT = new HandlerScheduler(new Handler(Looper.getMainLooper()), true)
callRequireNonNull(AndroidSchedulers$1)
Scheduler scheduler = AndroidSchedulers$1.call()
Scheduler scheduler = MainHolder.DEFAULT
Scheduler scheduler = new HandlerScheduler
对象2的 observeOn
observeOn(AndroidSchedulers .mainThread()) 其中AndroidSchedulers .mainThread() == AndroidSchedulers .MainHolder.DEFAULT
ObservableSubscribeOn 继承了Observable, 所以调用 Observable #observeOn(MainHolder.DEFAULT)
observeOn(scheduler, false, bufferSize())
对象2的$1 = new ObservableObserveOn<>(this, scheduler, delayError, bufferSize) this = 对象2
onAssembly(对象2的$1)
f == null
对象2的$1
对象3
对象2的$1
onNext参数：Activity#$2 = new Consumer<T>()
对象3的
subscribe(Activity#$2)
ObservableObserveOn 继承了Observable, 所以调用 Observable #subscribe(onNext), onNext = Activity#$2
subscribe(onNext, Functions.ON_ERROR_MISSING, Functions.EMPTY_ACTION)
ls = new LambdaObserver<>(onNext, onError, onComplete, Functions.emptyConsumer())
对象3的subscribe(ls)
onSubscribe(this, observer); this = 对象3 observer = ls
f == null
返回ls
observer = ls
对象3的 subscribeActual(ls)
alt
[scheduler instanceof TrampolineScheduler]
[else]
Scheduler.Worker w = scheduler.createWorker();
这里observer等于ls, source等于对象2
source.subscribe(new ObserveOnObserver<>(observer, w, delayError, bufferSize))
createWorker()
Scheduler.Worker w = HandlerWorker(hanlder, async)
new HandlerWorker(handler, async); handler, async 取值于构造器
对象3的$1 = new ObserveOnObserver(ls, w, delayError, bufferSize)
source.subscribe(对象3的$1) source等于对象2
对象2类型ObservableSubscribeOn继承了 Observable,所以调用的是 Observable的subscribe(Observer<? super T> observer 对象3的$1)
RxJavaPlugins.onSubscribe(this, observer) this = 对象2 observer = 对象3的$1
f == null
返回 对象3的$1
对象2的 subscribeActual(对象3的$1)
parent = new SubscribeOnObserver(对象3的$1) 同时parent实现了Observer<T>和 Disposable
observer.onSubscribe(parent); observer是对象3的$1
opt
[DisposableHelper.validate(this.upstream, d)]
this.upstream = d, upstream即是parent
opt
[d instanceof QueueDisposable]
queue = new SpscLinkedArrayQueue<>(bufferSize);
downstream.onSubscribe(this); this = 对象3的$1 downstream = 对象3的$1的第1个参数，即ls = new LambdaObserver<>(onNext, onError, onComplete, Functions.emptyConsumer()), onNext参数 = Activity$2
ls的onSubscribe(对象3的$1), 对象3的$1所实现的抽象类实现了 QueueDisposable
opt
[DisposableHelper.setOnce(this, d)]
onSubscribe.accept(this) onSubscribe = ls的第4个参数, 即Functions.emptyConsume()
什么事情也没做。
对象2的$2 = new SubscribeTask(parent) SubscribeTask类实现了Runnable
线程切换 scheduler.scheduleDirect(new SubscribeTask(parent)) scheduler取值于构造器, 等于io调度器
parent.setDisposable(scheduler.scheduleDirect(new SubscribeTask(parent)))
IoScheduler 继承了 Scheduler类, 所以 调用的是 Scheduler类 的scheduleDirect
scheduleDirect(run, 0L, TimeUnit.NANOSECONDS) run = 对象2的$2
w = createWorker() 创建了EventLoopWorker 持有 CachedWorkerPool池
decoratedRun = RxJavaPlugins.onSchedule(run)
task = new DisposeTask(decoratedRun, w)
异步执行 w.schedule(task, delay, unit) 即在io线程之中执行对象2的$2 调用者在当前线程,当前线程是 对象3的subscribe(Activity#$2) 调用者线程 即，Activity线程。
theadWorker = CachedWorkerPool池.get()
执行第1个参数run threadWorker .scheduleActual(run, delayTime, unit, tasks) tasks = new CompositeDisposable()
执行了 对象2的$2的run() 方法
异步 返回包装后的run, 类型ScheduledDirectTask, ScheduledDirectTask继承了 Disposable, 实现了 Callable<Void>
source.subscribe(parent) source = 对象1 取值于构造器
对象1的 .subscribe(parent) 对象1 = Observable$1
ObservableCreate继承了 Observable, parent实现了Observer<T>和Disposable 所以调用了 Observable.subscribe(parent), 即 subscribe(@NonNull Observer<? super T> observer)
observer = RxJavaPlugins.onSubscribe(this, observer) observe值不变 = parent
subscribeActual(parent) 当前对象是Observable$1
第2个局部parent = new CreateEmitter<>(observer)
observer.onSubscribe(第2个局部parent); observer = parent 这里的parent是最开始的parent, 如果把这里的parent 称为第2个局部parent
source.subscribe(第2个局部parent); source = Activity$1, 第2个局部parent实现了 ObservableEmitter和 Disposable, 即调用了 Activity$1实现的subscribe(第2个 局部parent)
Activity$1实现的subscribe里调用了 第2个局部parent.onNext(int); 第2个局部parent.onError(Throwable); 第2个局部parent.onComplete()
******** 线程切换 ******** 第2个局部parent.onNext(int) ******** 线程切换 ********
observer.onNext(int), observer等于最开始 的parent
downstream.onNext(int); downstream = 对象3的$1
对象3的$1.onNext(int) 对象3的$1 = new ObserveOnObserver(ls, w, delayError, bufferSize), 其中ObserveOnObserver实现了Runnable
opt
[done] return;
opt
[sourceMode != QueueDisposable.ASYNC] queue.offer(t);
schedule();
opt
安全判断 [getAndIncrement() == 0] worker.schedule(this);
这里的worker = HandlerWorker(handler, async) this = 对象3的$1
schedule(对象3的$1)
opt
[disposed] Disposable.disposed();
run = RxJavaPlugins.onSchedule(run);
Message message = Message.obtain(handler, scheduled);
message.obj = this; // Used as token for batch disposal of this worker's runnables.
opt
[async] message.setAsynchronous(true); 这里async = true，因此消息 通过handler传给了MainLooper, 线程切换到了主线程